

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Курсовой проект по курсу "Дискретный анализ"
Тема проекта "Эвристический поиск на графах"

Студент: А. Д. Галкин
Преподаватель: Н. К. Макаров
Группа: М8О-308Б-22
Дата: _____
Оценка: _____
Подпись: _____

Москва, 2025

Условие

Реализовать алгоритм A^* для неориентированного графа. Расстояние между соседями вычисляется как простое евклидово расстояние на плоскости.

Формат ввода:

- В первой строке даны два числа n и m ($1 \leq n \leq 10^4$, $1 \leq m \leq 10^5$) — количество вершин и рёбер в графе.
- В следующих n строках даны пары чисел $x \ y$ ($-10^9 \leq x, y \leq 10^9$), описывающие положение вершин графа в двумерном пространстве.
- В следующих m строках даны пары чисел в отрезке от 1 до n , описывающие рёбра графа.
- Далее дано число q ($1 \leq q \leq 300$) и в следующих q строках даны запросы в виде пар чисел $a \ b$ ($1 \leq a, b \leq n$) на поиск кратчайшего пути между двумя вершинами.

Формат вывода:

- В ответ на каждый запрос выведите единственное число — длину кратчайшего пути между заданными вершинами с абсолютной либо относительной точностью 10^{-6} , если пути между вершинами не существует, выведите -1 .

Методы решения

Код реализует алгоритм A^* для поиска кратчайшего пути в графе. В качестве эвристической функции используется евклидово расстояние между текущей вершиной и целевой вершиной. Алгоритм использует очередь с приоритетом (`priority_queue`) для выбора вершины с наименьшей оценкой $f(n) = g(n) + h(n)$, где $g(n)$ — стоимость пути от начальной вершины до текущей, а $h(n)$ — эвристическая оценка стоимости пути от текущей вершины до целевой.

Шаги алгоритма

1. Инициализация:

- Создать массивы для хранения стоимости пути (g_cost) и флага посещения ($visited$).
- Установить $g_cost[start] = 0$, остальные значения равны ∞ .
- Поместить начальную вершину в очередь с приоритетом с $f_cost = 0$.

2. Работа с очередью: Пока очередь с приоритетом не пуста:

- Извлечь вершину с наименьшим f_cost .
- Если текущая вершина совпадает с целевой, то путь найден.

3. Обработка соседей: Для каждого соседа текущей вершины:

- Вычислить $tentative_g = g_cost[u] + distance(u, v)$.
- Если $tentative_g < g_cost[v]$, обновить:
 - $g_cost[v]$ — минимальную стоимость пути до соседа.

– $f_cost = g_cost[v] + h(v)$, где $h(v)$ — эвклидово расстояние до цели.

- Добавить соседа в очередь с приоритетом, если он ещё не посещён.

4. Завершение:

- Если целевая вершина была извлечена из очереди, вывести её g_cost .
- Если очередь пуста, а путь не найден, вывести -1 .

Исходный код

```
#include <iostream>
#include <vector>
#include <queue>
#include <cmath>
#include <iomanip>
#include <limits>
using namespace std;

struct Edge {
    double distance;
    int vertex;

    Edge(double dist, int v) : distance(dist), vertex(v) {}
};

struct Node {
    double f_cost;
    int vertex;

    Node(double f, int v) : f_cost(f), vertex(v) {}

    bool operator>(const Node& other) const {
        return f_cost > other.f_cost;
    }
};

inline double euclidean_distance(pair<int, int>& p1, pair<int, int>& p2) {
    return sqrt(pow(p2.first - p1.first, 2) + pow(p2.second - p1.second, 2))
        ;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;

    vector<pair<int, int>> coordinates(n + 1);
    vector<vector<Edge>> graph(n + 1);

    for (int i = 1; i <= n; ++i) {
        cin >> coordinates[i].first >> coordinates[i].second;
    }

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
```

```

        double dist = euclidean_distance(coordinates[u], coordinates[v]);
        graph[u].emplace_back(dist, v);
        graph[v].emplace_back(dist, u);
    }

    int q;
    cin >> q;

    while (q--) {
        int start, goal;
        cin >> start >> goal;

        if (start == goal) {
            cout << fixed << setprecision(6) << 0.0 << '\n';
            continue;
        }

        priority_queue<Node, vector<Node>, greater<Node>> pq;
        vector<double> g_cost(n + 1, numeric_limits<double>::infinity());
        vector<bool> visited(n + 1, false);

        g_cost[start] = 0;
        pq.emplace(0.0, start);

        bool path_found = false;
        while (!pq.empty()) {
            Node current = pq.top();
            pq.pop();

            int u = current.vertex;

            if (visited[u]) continue;
            visited[u] = true;

            if (u == goal) {
                cout << fixed << setprecision(6) << g_cost[u] << '\n';
                path_found = true;
                break;
            }

            for (const Edge& edge : graph[u]) {
                int v = edge.vertex;
                double tentative_g = g_cost[u] + edge.distance;

                if (!visited[v] && tentative_g < g_cost[v]) {
                    g_cost[v] = tentative_g;
                    double h_cost = euclidean_distance(coordinates[v],
                                                         coordinates[goal]);
                    pq.emplace(g_cost[v] + h_cost, v);
                }
            }
        }

        if (!path_found) {
            cout << -1 << '\n';
        }
    }

    return 0;
}

```

Тест производительности

В процессе тестирования будем сравнивать наш алгоритм A* с Дейкстрой. Для тестирования рассмотрим насыщенные графы, разреженные, а также посмотрим на время работы при поиске пути к элементу, связи с которым нет.

```
alexey@alexey-Yoga-Slim-7-Pro-14IHU5:~/code/da-labs/cp/build$ ./benchmark <..
n = 10 m = 15
Dijkstra time: 0.000018 s
A* time: 0.000012 s
Dijkstra result: 1324.234190
A* result: 1324.234190
Dijkstra time: 0.000014 s
A* time: 0.000013 s
Dijkstra result: 1725.931403
A* result: 1725.931403
Dijkstra time: 0.000015 s
A* time: 0.000015 s
Dijkstra result: 3053.433784
A* result: 3053.433784
alexey@alexey-Yoga-Slim-7-Pro-14IHU5:~/code/da-labs/cp/build$ ./benchmark <..
n = 100 m = 150
Dijkstra time: 0.000162 s
A* time: 0.000139 s
Dijkstra result: 5264.315383
A* result: 5264.315383
Dijkstra time: 0.000112 s
A* time: 0.000062 s
Dijkstra result: 5941.390579
A* result: 5941.390579
Dijkstra time: 0.000093 s
A* time: 0.000072 s
Dijkstra result: 3491.785292
A* result: 3491.785292
alexey@alexey-Yoga-Slim-7-Pro-14IHU5:~/code/da-labs/cp/build$ ./benchmark <..
n = 1000 m = 4000
Dijkstra time: 0.000181 s
A* time: 0.000041 s
Dijkstra result: 1604.004397
A* result: 1604.004397
Dijkstra time: 0.000426 s
A* time: 0.000126 s
Dijkstra result: 2659.460141
A* result: 2659.460141
Dijkstra time: 0.000164 s
A* time: 0.000049 s
Dijkstra result: 2083.813527
A* result: 2083.813527
alexey@alexey-Yoga-Slim-7-Pro-14IHU5:~/code/da-labs/cp/build$ ./benchmark <..
n = 100 m = 100
Dijkstra time: 0.000117 s
```

```
A* time: 0.000125 s
Dijkstra result: -1.000000
A* result: -1.000000
Dijkstra time: 0.000066 s
A* time: 0.000054 s
Dijkstra result: 3746.421760
A* result: 3746.421760
Dijkstra time: 0.000039 s
A* time: 0.000035 s
Dijkstra result: 2691.991992
A* result: 2691.991992
```

Как можно заметить, алгоритм A^* на всех тестах показывает себя лучше, чем алгоритм Дейкстры.

Выводы

В данной курсовой работе был реализован алгоритм A^* для поиска кратчайшего пути на графе. Было изучено применение эвристической функции для оптимизации поиска и использования очереди с приоритетом для эффективного выбора следующей вершины для обработки. Реализация алгоритма позволяет находить кратчайшие пути между заданными вершинами с высокой точностью.